

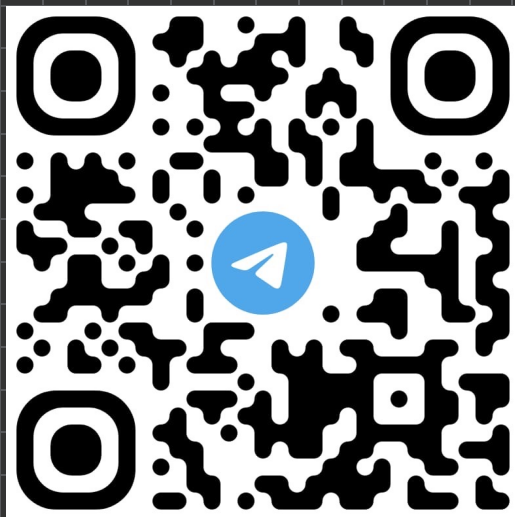
(LEC - 36)

How JS execute Internally ?

↳ Execution Context →

↳ Environment → Jaha per JS execute hoti hai

Telegram



Discord



Execution context

2 phase

1 creation ✓

2 Execution ✓

1 creation phase

memory creation / allocation

↳ memory reserve

declaration

var a;

let

const

function fun1() {

}

const f = fun() {

}

const n = () => {

arrow
func. clo.

}

key : value

① var a = 9;

a : undefined

↳ initialize with undefined in var

② let/const

let b = 10

const c = 11

b : <value unavailable>

c :

↳ value is uninitialized in let/const

③ function fun1() { | fun1() : { ... }
}

↳ In case of fun declaration
entire function stored in memory

④ var fun2 = function() { | Exactly same as
var variable
or
fun2 : undefined
var fun2 = () => {
}

⑤ let/const fun3 = function() {
}

or

let/const fun3 = () => {
}

same as
let/const variable
rule

fun3 : <value unvariable>

2 Execution → Top to bottom
line by line

→ assigning values

→ calling function

→ evaluating expressions

Execution context

2 types

1

Global Execution context

→ one time creation

2

function Execution context

↳ create on every function call

Debugger

↳ custom breakpoints

```
var a = 5 ; ←  
let b = 10;  
console.log(a + b); x
```

Execution context → global

creation	Execution
a: 5 b: 10	a = 5 b = 10 log(5 + 10) ↓ 15

```
var a = 5 ;
let b = 10;
console.log(a + b); ✕
```

```
→ function fun1(){
  let num1 = 10 ;
  let num2 = 20;
  return num1 + num2;
}
```

```
const result = fun1() ←
console.log(result); ←
```

Execution context → global

creation

a: 5
b: 10

fun1: { ... }

result: 30

Execution

Execution context → fun1	
creation	Execution
num1: 10	num1: 10
num2: 20	num2: 20
	return num1 + num2
	return 30

log(result)
30

```
function fun1(){  
  → let num1 = 10 ;  
    let num2 = 20;  
    return num1 + num2;  
}
```

Execution Context → fun1

creation		Execution	
num1:	10	num1 = 10	
num2:	20	num2 = 20	
		return num1 + num2	
		return 30	

```

var a = 5;
let b = 10;
console.log(a + b);

function outer() {
  let num1 = 10;
  let num2 = 20;

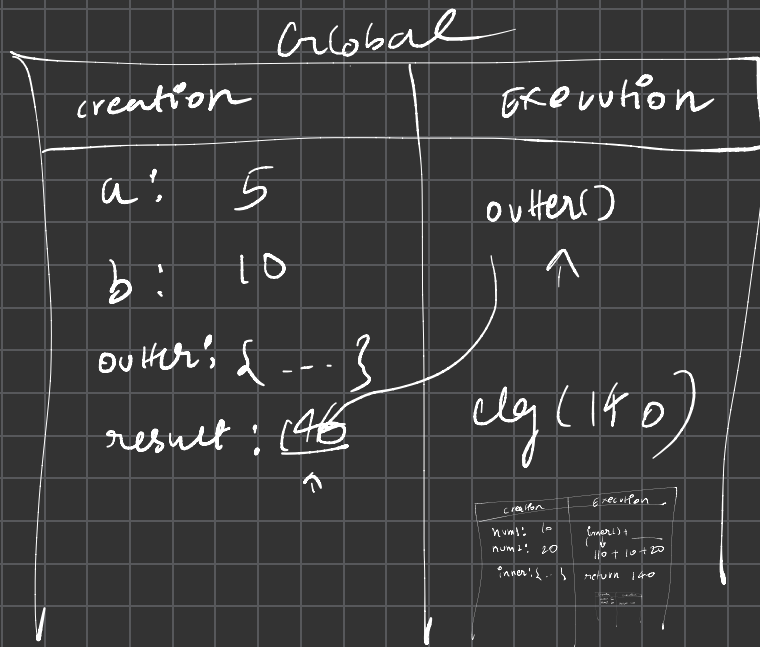
  function inner() {
    let num1 = 50;
    let num2 = 60;
    return num1 + num2;
  }

  const result = inner() + num1 + num2;

  return result;
}

const result = outer();
console.log(result);

```



creation	Execution
num1: 10	inner() +
num2: 20	↓
	110 + 10 + 20
inner: { - }	return 140

creation	Execution
num1: 50	
num2: 60	return 110

Creation

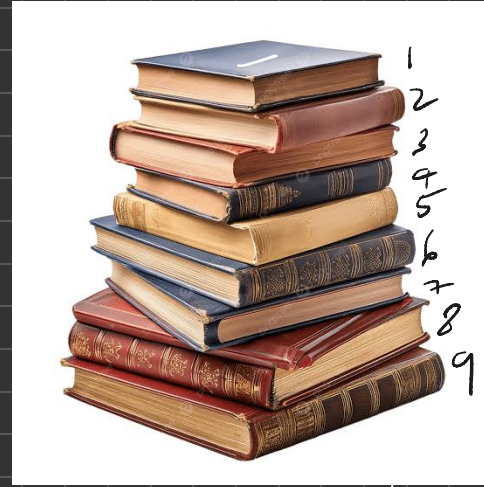
num1: 50

num2: 60

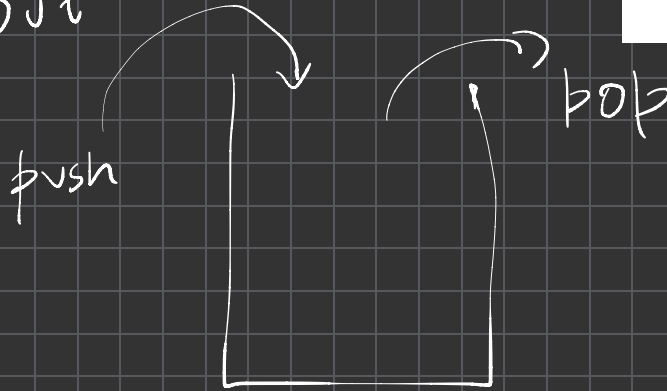
Execution

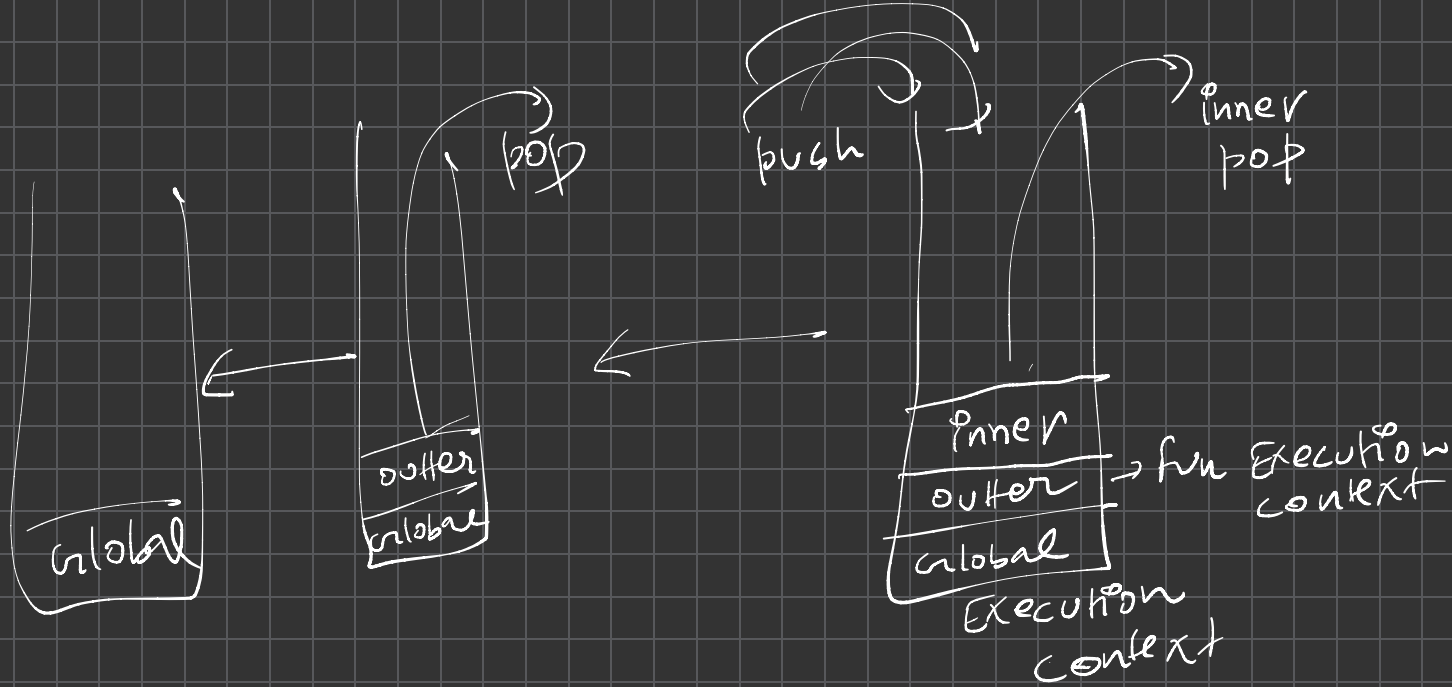
← return 110

call stack
→
dsa



LIFO
↓
Last in
first out





clg(fun2) → undefined

```
fun2() ✓
```

```
var fun2 = function(){  
  console.log("hello");  
}
```

→ fun2: undefined

fun2()

↓

undefined()

TypeError

fun2 is not a function

```
recurse()
```

[illegible]

$Tdz \rightarrow \text{let } x \text{ const}$